

```

int m25p80_read(struct mtd_info *mtd, loff_t from,
size_t len, size_t *retlen, u_char *buf)
{
    struct m25p *flash = mtd_to_m25p(mtd);
    struct spi_transfer t[2];
    struct spi_message m;

    spi_message_init(&m);
    memset(t, 0, (sizeof t));

    /* setup CMD transfer */
    t[0].tx_buf = flash->command;
    t[0].len = 4; /* CMD size */
    flash->command[0] = OPCODE_READ;
    flash->command[1] = from >> 16;
    flash->command[2] = from >> 8;
    flash->command[3] = from;
    spi_message_add_tail(&t[0], &m);

    /* setup RSP transfer */
    t[1].rx_buf = buf;
    t[1].len = len;
    spi_message_add_tail(&t[1], &m);
    /* send command to SPI framework */
    spi_sync(flash->spi, &m);

    *retlen = m.actual_length - 4; /* don't count cmd
size */
    return 0;
}

```

Now that it is clear how a chip driver creates SPI messages and SPI transfers, and how those messages reach the SPI master, let's look at real data transfer.

The transfer method of SPI controller drivers:

The transfer method of the SPI controller driver is called whenever any operation (read/ write/ erase/ status) is performed on the slave device. As explained earlier, this method receives SPI messages as an argument. The transfer method has the responsibility of processing this SPI message and carrying out the appropriate operation on the SPI slave device. However, this method has a constraint in that it cannot sleep. Therefore, it is very common that this method only offloads the work to a work-queue and returns. The work-queue function later completes the actual I/O.

The actual transfer

The work-queue function responsible for the actual transfer is SPI slave dependent. You need to refer to the SPI slave hardware manual to understand how exactly the data transfer can be done. Some SPI slave devices are interrupt driven, and are, therefore, capable of informing about I/O completion through an interrupt; while some simple devices have to be polled for completion.

The following transfer function shows a typical data transfer from a Winbond flash chip W25Q64BV. This chip is controlled by some registers that can be memory mapped by the PCI driver.

- Read the 'Software Sequencing Flash Control Register' to check if the slave device is free
- Write the I/O address to the 'Flash Address Register'
- Write the operation code (SPI opcode) to the 'Opcode Type Configuration Register'
- Write opcode meta data to the 'Opcode Menu Configuration Register'
- Write opcode prefix data to the 'Prefix Opcode Configuration Register'
- Write data to 'Flash Data Registers'
- Trigger operations by writing to the 'Software Sequencing Flash Control Register'
- Read the 'Software Sequencing Flash Control Register' to check if the operation is completed
- Read data from the 'Flash Data Registers'

As you see, this function has to perform chip-specific work. Some steps in the above list may or may not be applicable, based on the SPI opcode under process.

As described, there are two important things an SPI controller driver should do. The first is to implement chip-specific setup/ transfer/ clean-up functions, and the second is to register with the Linux device/MTD/SPI sub-system. The first part is hardware specific while the second part is common.

So far, this article has not mentioned the locking and concurrency aspects. But it is very important to consider them when working on SMP systems or working on a system with more than one SPI slave device. There are two important cases that a driver author should keep in mind.

1. The SPI controller driver should synchronise concurrent requests to multiple SPI devices on the same bus. If two chip drivers issue requests at the same time to two devices on the same SPI bus, then the controller driver needs to guarantee that one request waits until the other has finished. This kind of synchronisation is usually implemented by the queuing mechanism within the controller driver. The common queue represents a unique resource that is protected via some sort of locking. Only the driver that owns the lock can add new requests to the queue, others will have to wait until the lock is released. Therefore, it is very common to see work-queue offloading and a queue in the transfer method.
2. The setup function is called as soon as any SPI slave device is found. Therefore, it is possible that when the setup for a new SPI slave device is called, the transfer method for some old SPI slave device is in progress. Hence, the setup method must not update any shared data structures. 

By: Mohan Lal Jangir

The author, whose interests are the Linux kernel, data networking, network processors and network security, is a senior technical manager at Samsung Research India, Bengaluru, and holds a Masters from IIT Delhi.